

PE6201 · Class 1 · Capsule 3 — Live Demo

Why it can't count the letters in "strawberry"

Emerging AI Technologies · MSc in Enterprise Artificial Intelligence · NTU

Cells 1–5 need no API key — just `pip install tiktoken` . Cell 6 calls a real model.

Run them in sequence. Later cells depend on variables defined earlier.

Runtime: about 45 seconds.

CELL 1 — Setup

Downloads the tokenizer used by the GPT-4o / GPT-5 family. Run once.

```
In [11]: !pip install tiktoken
```

```
Requirement already satisfied: tiktoken in /Users/avs/Desktop/AI/tAIEEnv/lib/python3.14/site-packages (0.13.0)
Requirement already satisfied: regex in /Users/avs/Desktop/AI/tAIEEnv/lib/python3.14/site-packages (from tiktoken) (2026.7.19)
Requirement already satisfied: requests in /Users/avs/Desktop/AI/tAIEEnv/lib/python3.14/site-packages (from tiktoken) (2.33.1)
Requirement already satisfied: charset_normalizer<4,>=2 in /Users/avs/Desktop/AI/tAIEEnv/lib/python3.14/site-packages (from requests->tiktoken) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /Users/avs/Desktop/AI/tAIEEnv/lib/python3.14/site-packages (from requests->tiktoken) (3.11)
Requirement already satisfied: urllib3<3,>=1.26 in /Users/avs/Desktop/AI/tAIEEnv/lib/python3.14/site-packages (from requests->tiktoken) (2.6.3)
Requirement already satisfied: certifi>=2023.5.7 in /Users/avs/Desktop/AI/tAIEEnv/lib/python3.14/site-packages (from requests->tiktoken) (2026.2.25)
```

```
In [12]: import os
        from dotenv import load_dotenv

        # Force-load the .env file from the current directory
        load_dotenv(override=True)
```

Out[12]: True

```
In [13]: #!pip install tiktoken --quiet

import tiktoken
from IPython.display import HTML, display

ENC = tiktoken.get_encoding("o200k_base") # the GPT-4o / GPT-5 family encoding

PALETTE = ["#D71440", "#0054A6", "#00875A", "#B54708",
           "#6941C6", "#0F327C", "#B42318", "#027A48"]

def pieces_of(text):
    """Split text into the exact chunks the model receives, with their IDs."""
    ids = ENC.encode(text)
    return [ENC.decode([i]) for i in ids], ids

def show_tokens(text, title=""):
    pieces, ids = pieces_of(text)
    spans = []
    for i, p in enumerate(pieces):
        shown = p.replace(" ", " ").replace("\n", "↵")
        c = PALETTE[i % len(PALETTE)]
        spans.append(
            f'<span style="display:inline-block;background:{c};color:#fff;'
            f'font-family:Menlo,Consolas,monospace;font-size:30px;font-weight:600;'
            f'padding:10px 14px;margin:4px;border-radius:8px;">{shown}</span>')
    head = (f'<div style="font-family:Arial;font-size:15px;color:#5A5A5A;'
            f'margin:14px 0 4px;">{title}</div>') if title else ""
    display(HTML(head + '<div style="line-height:1.9;">' + "".join(spans) + '</div>'))
    return pieces, ids

print("Ready.")
```

Ready.

CELL 2 — The word

Ask the room first: *how many pieces do you think this becomes?*

The second output is the one that matters — the raw token IDs. There are no letters in them at all.

```
In [22]: # Ask the room first: "How many pieces do you think this becomes?"

WORD = "strawberry" #assassination, mississippi, chrysanthemum

print(f'"{WORD}" is {len(WORD)} letters long.\n')
pieces, ids = show_tokens(WORD, "First, the chunks it is broken into:")

print(f"\n{len(pieces)} tokens: {pieces}")

# --- the part that actually makes the point ---
display(HTML(
    f'<div style="font-family:Arial;font-size:15px;color:#5A5A5A;margin:18px 0 6px;">'
    f'But the model does not receive text at all. It receives these:</div>'
    f'<div style="font-family:Menlo,Consolas,monospace;font-size:38px;font-weight:700;'
    f'color:#FFFFFF;letter-spacing:2px;">{ids}</div>'
    f'<div style="font-family:Arial;font-size:17px;margin-top:14px;max-width:780px;">'
    f'Three numbers. There is no <b>"r"</b> anywhere in that – there are no letters at all. '
    f'Whatever the model knows about how <i>{WORD}</i> is spelled, it had to learn as a '
    f'fact <i>about</i> those numbers. Nothing in training forced it to.</div>'))
```

"strawberry" is 10 letters long.

First, the chunks it is broken into:



3 tokens: ['st', 'raw', 'berry']

But the model does not receive text at all. It receives these:

Three numbers. There is no "r" anywhere in that — there are no letters at all. Whatever the model knows about how *strawberry* is spelled, it had to learn as a fact *about* those numbers. Nothing in training forced it to.

CELL 3 — Why counting breaks

The model is not counting — it is recalling a plausible association.

```
In [16]: LETTER = "r"

rows = "".join(
    f'<tr>'
    f'<td style="padding:6px 14px;font-family:Menlo,monospace;font-size:22px;">'
    f'<span style="background:{PALETTE[i%len(PALETTE)]};color:#fff;'
    f'padding:4px 10px;border-radius:6px;">{p}</span></td>'
    f'<td style="padding:6px 14px;font-family:Arial;font-size:18px;color:#5A5A5A;">'
    f'contains {p.lower().count(LETTER)} &times; "{LETTER}"</td>'
    f'</tr>'
    for i, p in enumerate(pieces))

display(HTML(
    f'<div style="font-family:Arial;font-size:15px;color:#5A5A5A;margin:14px 0 6px;">'
    f'To answer "how many {LETTER}\s?", the model has to reason across these:</div>'
    f'<table style="border-collapse:collapse;">{rows}</table>'
    f'<div style="font-family:Arial;font-size:17px;margin-top:14px;max-width:780px;">'
    f'You can do this because you can <i>see</i> the letters. The model cannot – '
    f'to it these are the opaque IDs <b>{ids}</b>. It is not counting; it is recalling '
    f'a plausible association. <i>"berry"</i> has a memorable double-r, and '
    f'<i>berry / cherry / merry</i> all have exactly two – so <b>2</b> is the '
    f'well-worn answer, and the lone r hiding in <i>"str"</i> gets dropped.</div>')
```

```
f'<div style="font-family:Arial;font-size:15px;color:#5A5A5A;margin-top:12px;'
f'max-width:780px;"><b>Worth noting:</b> reasoning models often get this right now – '
f'because they spell the word out letter by letter first. That turns "recall a fact '
f'about a token" into "read a list I just wrote." Same tokenizer, different '
f'strategy.</div>''))
```

To answer "how many r's?", the model has to reason across these:

ch	contains 0 × "r"
rys	contains 1 × "r"
anthem	contains 0 × "r"
um	contains 0 × "r"

You can do this because you can see the letters. The model cannot — to it these are the opaque IDs **[332, 35960, 108893, 394]**. It is not counting; it is recalling a plausible association. *"berry"* has a memorable double-r, and *berry / cherry / merry* all have exactly two — so **2** is the well-worn answer, and the lone r hiding in *"str"* gets dropped.

Worth noting: reasoning models often get this right now — because they spell the word out letter by letter first. That turns "recall a fact about a token" into "read a list I just wrote." Same tokenizer, different strategy.

CELL 4 — The same problem, in other languages

Tokenisation is not language-neutral.

In [18]: *# Directly relevant: tokenisation is not language-neutral, and you pay per token.*

```
SAMPLES = {
    "English": "Artificial intelligence is changing how we work.",
    "Chinese": "",
    "Hindi": "आर्टिफिशियल इंटेलिजेंस हमारे काम करने के तरीके को बदल रहा है।",
```

```

    "Indonesian": "Kecerdasan buatan mengubah cara kita bekerja.",
}

print(f"{'Language':12s} {'Characters':>11s} {'Tokens':>8s} {'Chars/token':>12s}")
print("-" * 47)
for lang, text in SAMPLES.items():
    n_tok = len(ENC.encode(text))
    print(f"{'lang':12s} {'len(text)':>11d} {'n_tok':>8d} {'len(text)/n_tok:>12.2f}")

print("\nSame sentence, same meaning – very different token counts.")
print("You are billed per token. Some languages cost several times more to process.")

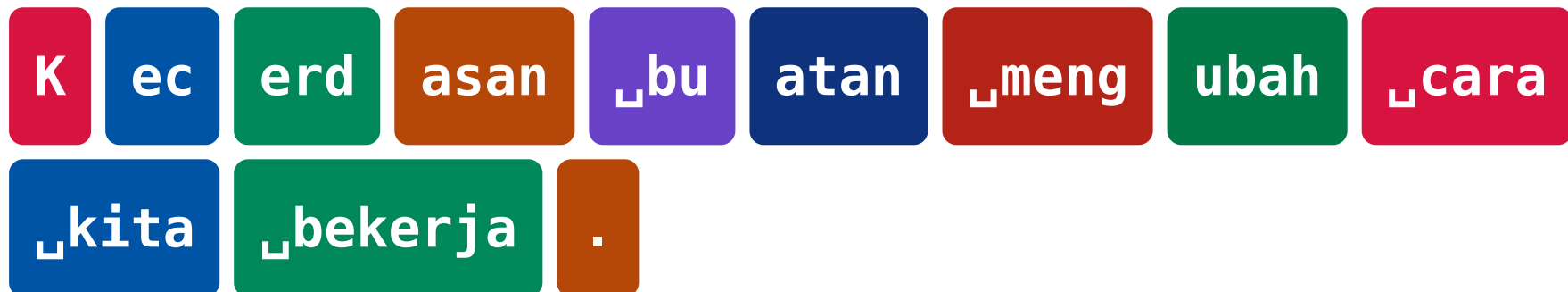
_ = show_tokens(SAMPLES["Indonesian"], "The Hindi sentence, tokenised:")

```

Language	Characters	Tokens	Chars/token
English	48	8	6.00
Chinese	16	8	2.00
Hindi	61	20	3.05
Indonesian	45	12	3.75

Same sentence, same meaning – very different token counts.
 You are billed per token. Some languages cost several times more to process.

The Hindi sentence, tokenised:



CELL 5 — What that costs you

The economics of the same fact.

The slide claims tokenisation is an economic constraint, not just a technical curiosity. This is the arithmetic.

Compares **input cost only** — what it costs the model just to *read* the text. Output cost is identical across languages, so including it would hide the effect entirely.

```
In [23]: # The slide claims tokenisation is an economic fact, not just a technical one.
# This is the arithmetic behind that claim.

RATE_IN = 0.15 / 1_000_000      # USD per input token – illustrative; rates vary and change often

# --- 1. What one document costs to process ---
DOC = SAMPLES["English"] * 400      # roughly a 3,000-word document
n_doc = len(ENC.encode(DOC))
print(f"A ~3,000 word document -> {n_doc:,} tokens -> ${n_doc*RATE_IN:.4f} to read once")
print(f"Across a million documents: ${n_doc*RATE_IN*1_000_000:,.0f}")
print()

# --- 2. The same content, in four languages ---
# Comparing INPUT cost only – what it costs the model just to READ the text.
# (Output cost is identical across languages, so including it would hide the effect.)
DOCS = {lang: text*40 for lang, text in SAMPLES.items()}

print(f"{'Language':12s} {'Characters':>11s} {'Tokens':>8s} {'Per 1M docs':>13s} {'vs English':>12s}")
print("-" * 60)
base = None
for lang, text in DOCS.items():
    t = len(ENC.encode(text))
    cost = t * RATE_IN * 1_000_000
    if base is None: base = cost
    print(f"{lang:12s} {len(text):>11,} {t:>8,} {cost:>12,.0f}$ {cost/base:>11.2f}x")

print()
print("Same meaning. Same work. Different bill – purely because of how the text is split.")
print("Tokenisation is not only why it cannot count. It is also what you are charged for.")
```

A ~3,000 word document -> 3,200 tokens -> \$0.0005 to read once
Across a million documents: \$480

Language	Characters	Tokens	Per 1M docs	vs English
English	1,920	320	48\$	1.00x
Chinese	640	320	48\$	1.00x
Hindi	2,440	800	120\$	2.50x
Indonesian	1,800	441	66\$	1.38x

Same meaning. Same work. Different bill – purely because of how the text is split.
Tokenisation is not only why it cannot count. It is also what you are charged for.

CELL 6 — Optional — watch a real model try

Needs an OpenRouter key. `strawberry` is now often answered correctly; `chrysanthemum` or `onomatopoeia` fail more reliably.

```
In [24]: !pip install python-dotenv
```

Requirement already satisfied: python-dotenv in /Users/avs/Desktop/AI/tAIEEnv/lib/python3.14/site-packages (1.2.2)

```
In [25]: # Needs an OpenRouter key. Skip if offline – Cells 1–5 make the point already.
# Note: popular models may now answer "strawberry" correctly; it became famous
# enough to be trained on. Try a rarer word to see the failure reliably.

RUN_LIVE = True          # set True in class if the key is loaded

import os
from dotenv import load_dotenv

# Force-load the .env file from the current directory
load_dotenv(override=True)

API_KEY = os.environ.get("MY_PRIVATE_OPENROUTER_KEY") # os.environ.get("OPENROUTER_API_KEY").. my private open rout
#print(API_KEY)
```

```
In [26]: # Quick test to confirm it works
if API_KEY:
    # Print the key length and masked characters for safety
    print(f"Key successfully loaded! (Length: {len(API_KEY)}, Starts with: '{API_KEY[:7]}...')")
```

```
else:
    print("Key is NOT detected in Python.")
```

Key successfully loaded! (Length: 73, Starts with: 'sk-or-v...')

```
In [33]: #API_KEY = "" # os.environ.get("OPENROUTER_API_KEY")
MODEL = "openai/gpt-4o-mini"
TEST_WORD = "strawberry" # try also: "chrysanthemum", "Mississippi", "onomatopoeia"
if RUN_LIVE and API_KEY:
    import json, urllib.request
    req = urllib.request.Request(
        "https://openrouter.ai/api/v1/chat/completions",
        data=json.dumps({
            "model": MODEL,
            "messages": [{"role": "user",
                           "content": f'How many letter "r"s are in "{TEST_WORD}"? ' #change char here
                           f'Answer with just the number.'}],
        }).encode(),
        headers={"Authorization": f"Bearer {API_KEY}",
                 "Content-Type": "application/json"})
    try:
        with urllib.request.urlopen(req, timeout=25) as r:
            answer = json.load(r)["choices"][0]["message"]["content"].strip()
            truth = TEST_WORD.lower().count("r") #change char here
            print(f'Model says: {answer} | Actually: {truth}')
            print("MATCH" if str(truth) in answer else "WRONG – and confidently so.")
    except Exception as e:
        print("Live call failed – use the fallback screenshot.", e)
else:
    print("Live call skipped. Cells 1–5 already show the mechanism.")
```

Model says: 2 | Actually: 3
WRONG – and confidently so.